

第 1 週：設計的基本概念

本週一句話：Verilog 不是在寫「步驟」，是在畫「電路」。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week01_basics
```

每個範例都是這三條（外加一條檢查），把 `01_gates` 換成你要跑的名字：

```
iverilog -o sim.out 01_gates.v 01_gates_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_gates.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_gates`
- `02_half_adder`
- `03_full_adder`

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 說得出「硬體描述語言」跟「程式語言」差在哪
- [] 看得懂 `module` / 腳位 / `endmodule` 的骨架

- [] 會用 `assign` 寫組合邏輯
- [] 認得 `& | ~ ^` 四個位元運算子
- [] 會把小模組**實例化**成大模組 (本週最重要)
- [] 能自己跑完 `sim` 三步驟並看懂波形

一、最重要的觀念：同時 vs 依序

寫軟體：

```
a = 1;      // 先做這行  
b = a + 1;  // 再做這行
```

寫 Verilog：

```
assign y1 = a & b;    // 這三行  
assign y2 = a | b;    // 全部  
assign y3 = a ^ b;    // 同時發生
```

這三行沒有先後順序。把它們上下對調，電路完全一樣。

因為它們描述的是三條實體的線，三個實體的閘——電流不會排隊，它們本來就是同時在導通。

[!IMPORTANT] 每次你看 Verilog 覺得「怪怪的」，就回來想這句：你不是在下指令，你是在描述一堆同時存在的東西。

二、module 骨架

```
module 名字 (  
    input wire 進來的腳,  
    output wire 出去的腳    // ← 最後一個不能有逗號  
);  
  
    // 裡面是這顆零件的行為  
  
endmodule                // ← 沒有分號
```

對照實體：



關鍵字	意思
module	定義一顆零件
input / output	這隻腳的方向
wire	一條線，沒有記憶
assign	連續指定：右邊一變，左邊立刻跟著變

三、四個位元運算子

符號	名稱	什麼時候是 1
&	AND	兩個都是 1
	OR	至少一個是 1
~	NOT	輸入是 0
^	XOR	兩個不一樣

^ (XOR) 最容易忘，但它是加法的核心，一定要記熟。

四、本週三個範例

01_gates — 四個基本閘

```
iverilog -o sim.out 01_gates.v 01_gates_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_gates.gtkw
```

看什麼：波形上四條輸出線，隨著 a、b 變化「立刻」跟著變——沒有時脈、沒有延遲、沒有記憶。這就是**組合邏輯**。

終端機會印出真值表，跟波形對照著看。

02_half_adder — 半加器

```
iverilog -o sim.out 02_half_adder.v 02_half_adder_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_half_adder.gtkw
```

看什麼：**sum** 就是 XOR，**carry** 就是 AND。

1 + 1 = 2，但一個位元裝不下 2，所以要「進位」—— **carry=1, sum=0** 合起來讀就是二進位的 **10**，也就是十進位的 2。

這個測試會**自己判斷對錯**，最後印「全部正確！」。這是第一次接觸 self-checking testbench，第 5 週會深入。

03_full_adder — 全加器 ★ 本週重點

```
iverilog -o sim.out 03_full_adder.v 03_full_adder_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_full_adder.gtkw
```

看什麼：這個檔裡面有兩個 module。full_adder 用了兩顆 ha。

```
ha ha1 (  
    .a      (a),          // 左邊 = ha 的腳位名  
    .b      (b),          // 右邊 = 我這邊的線名  
    .sum     (s1),  
    .carry   (c1)  
);
```

這叫**實例化** (instantiation) —— 就是「從零件盒拿一顆出來，插到板子上，接好線」。

[!IMPORTANT] 這是整個數位設計的核心技能。從今以後所有大電路都是這樣蓋的：小模組 → 中模組 → 大模組。CPU 也是這樣一層一層疊出來的。

打開 GTKWave 後，注意左邊的樹會有三層：

```
▼ full_adder_tb  
  ▼ uut                ← full_adder  
    ha1                 ← 第一顆半加器  
    ha2                 ← 第二顆半加器
```

點進 **ha1**，你可以看到它內部的訊號。這就是階層式設計 (第 10 週主題) 的第一次體驗。

五、怎麼跑

1. 回到 **VERI** 資料夾，**雙擊** **env.bat**
2. 在跳出來的視窗打：

```
cd week01_basics  
iverilog -o sim.out 01_gates.v 01_gates_tb.v  
echo %ERRORLEVEL%  
vvp sim.out  
gtkwave 01_gates.gtkw
```

只想看文字不想開波形 → 省略最後那行 **gtkwave** 就好。

忘記有哪些範例 → 打 **dir *.v** 看這個資料夾有什麼設計檔。

六、練習題

做完再往下一週。改完存檔，重跑 `sim` 就好。

練習 1 (暖身)

在 `01_gates.v` 加一個 NAND 輸出：

```
output wire y_nand
```

```
assign y_nand = ~(a & b);
```

記得 `tb` 也要加對應的 `wire` 和接線。跑跑看波形上 NAND 是不是 AND 的相反。

練習 2 (重要)

不看 `03_full_adder.v`，自己寫一個 2 位元加法器 `adder2`：

```
輸入： a[1:0], b[1:0]  
輸出： sum[1:0], cout
```

提示：用兩顆 `full_adder` 串起來，第一顆的 `cout` 接到第二顆的 `cin`，第一顆的 `cin` 接 `1'b0`。

這就是所謂的漣波進位加法器 (ripple carry adder)，也是你 `counter.v` 裡 `count + 1` 被綜合出來的東西。

練習 3 (思考，不用寫)

`03_full_adder` 的波形上，`cout` 和 `sum` 有沒有「延遲」？為什麼？跟你的 `counter` 的波形比起來，最大的差別是什麼？

► 看答案

七、本週檢核

跑得出來、看得懂，就可以進第 2 週：

- [] 三個範例都跑過，波形都看過
- [] 說得出 `assign` 和 `always @(posedge clk)` 的差別
- [] 練習 2 寫出來而且測試通過
- [] 在 GTKWave 左邊的樹點得進 `ha1` 看內部訊號

卡住的話，回頭看 [../explain.md](#) 的語法急救表。

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。